Eclipse: a multi-agent simulator for election manipulation research

Project report: simulator, configuration, studies, results and certified equilibria

Repository `splittersplatter/Multi_Agent_System` · state at commit `86de48b` (9 November 2026) · All results in this report were re-run from that commit for this report (Python 3.13, 4-core Linux container).

Abstract. Eclipse is one cumulative, deterministic agent-based simulator of repeated plurality elections. Voters hold private ranked preferences, move in a 2-D space, observe neighbours through directed graph layers, exchange delayed reports and vote with one of 26 built-in rules (or a Python plugin). It implements the seven manipulation mechanisms of the Eclipse-MAS proposal: dynamic proximity, coalitions, campaign edges, public-poll distortion, spatial nomination, alignment attacks with an aggregate detector, and hidden leader adaptation. A 17-arm matrix composes all seven. One engine drives a CLI, a browser UI with rewind and edit, checkpoint replay and exports. This report defines the configuration language and summarises the studies. It reports results re-generated for this report: 13 voting heuristics, 7 mechanism comparisons, the combined matrix and a majority-estimation pilot. It also covers the convergence guarantees (F1–F5) and exact optima from the built-in oracles. The full suite of **343 tests passes**, and **every study run completed with zero failures**.

6.0k	**6.9k**	**26**	**7**	**31**	**5**
lines of engine Python (+0.5k web UI)	lines of tests · 343 passing	built-in voting rules + plugins	manipulation mechanisms + combined matrix	example TOML scenarios, all replay-verified	certified convergence domains (F1–F5)

1. Goals and scope

The project asks how far bounded, budgeted manipulators can change a repeated election when voters only see local and possibly corrupted information. The roadmap has 12 stages. Stage 00 is the baseline and research contract; Stage 01 is the simulator; Stage 02 adds epistemics. Stages 03–09 add one mechanism each, Stage 10 integrates them, and Stage 11 delivers experiments. Stage 00 is verified and Stages 01–10 are implemented, with formal acceptance gates still open. Stage 11 (frozen, held-out experiments) is pending. Three invariants run through the design. Private preferences, cast ballots, reports and perceptions are always stored separately. Every manipulation spends from an explicit, logged budget. Knowledge is factive, so a false poll is an observation and never a truthful announcement.

2. Architecture

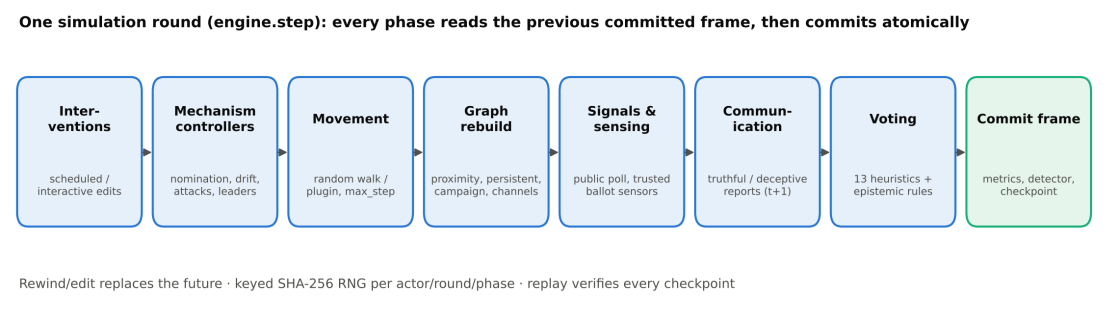


Figure 1. Phase order of a round. Reports generated at round *t* describe round *t−1* and arrive at *t+1*. Each actor/round/phase has its own keyed RNG stream, so policy choices never perturb movement and paired controls stay matched.

Layer	Modules (responsibility)
Configuration/state	`model.py` canonical TOML parsing & immutable schemas; `nomination`, `alignment`, `leaders`
Round coordinator	`engine.py`: phase order, atomic frame commit, rewind/resume
Observations	`observations` (trusted sensors), `coalitions` (scoped sight/interception), `policies` (plugin sandbox of inputs)
Decisions	`voting`, `epistemic`, `network`, `campaign`, `signals`, `detector`, `majority`
Replay/provenance	`records`, `provenance`: SHA-256 of core, pipeline, browser assets, plugins
Evaluation	`metrics`, `*_experiments`, `combined`, `majority_experiments`
Delivery	`cli` (18 commands), `visual` + `web/*`, `exports`, `render` (CSV/SVG/PNG/GIF)

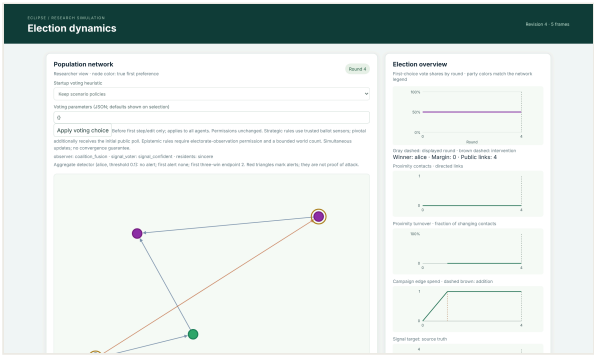


Figure 2. Browser UI running `all-enabled.toml` at round 4: population network with persistent (blue) and purchased campaign (orange) edges. Also shown are vote-share, contact, turnover and campaign-spend charts, plus the detector status line.

3. Simulator configuration definitions

A scenario is one TOML file. Top-level keys are `parties` (also the deterministic tie order), `k` (global ballot length, $1 \le k \le |C|$), `seed` and `rounds`. The election rule is first-choice plurality over exactly-k distinct ranked ballots, and round 0 is sincere. Validation is canonical: the parser re-checks every `Scenario`, including dataclass replacements. Invalid input raises `ElectionError` before any state is created.

Table / field	Definition and bounds
`[groups.<name>]`	`voting` (26 built-ins or `python:mod:fn`), `movement` (`stationary` | `random_walk` | plugin), `max_step` $\in [0,1]$, `communication` (`none` | `truthful` | `target_report` | plugin), `graph`, `proximity_radius` $\in [0,\sqrt{2}]$, `parameters`, `capabilities`
`capabilities`	15 booleans. Sensing, sight, interception, public-signal and roster permissions default to *false*: `sense_public_ballots`, `sense_private_ballots`, `observe_target_preferences`, `intercept_private_reports`, `observe_public_signal`, `observe_electorate`. Send/receive/move/modify permissions default to *true*.
`[[people]]`	Unique `id`, `group`, strict full `preference` (or `ideal` $\in [0,1]^d$ in spatial scenarios), `position` $\in [0,1]^2$, per-person `overrides`
`[network]`	`radius`; `proximity_mode` = dynamic | `frozen` | `static`; directed `persistent_edges`, `campaign_edges`; `private_channels`. Proximity arcs are sender→receiver whenever the sender lies within the receiver's radius.
`[[interventions]]`	Scheduled edits at a `round` for a `person`: preference, position, group, overrides, one-round ballot, edge/channel add/remove
`[[coalitions]]`	`members`, `target`; per-round/total budgets for false reports, preference sight (`sight_targets`) and private-channel eavesdropping (`eavesdrop_channels`)
`[campaign_controller]`	`target`, `policy` = none | random | `target_heuristic`, `lifetime_budget`, `per_round_budget`, `max_candidates`; each purchased directed edge costs 1 and cannot be repurchased
`[signal_controller]`	`target`, `policy` = truthful | bounded_random | target_biased, `schedule` = initial | repeated; cost = L1(truth, report)/2 per broadcast and per episode
`[nomination]`	`m` nominees from `eligible`; `mode` = endogenous (nearest eligible incoming neighbour or self) | fixed | random
`[alignment]`, `.attack`	`drift_max_step`; attack `target`, `targets`, `policy` = none | toward_target | random, `per_person_round`, `total_budget` (L2 path length)
`[detector]`	Passive: alert iff Δ target share between rounds > `threshold`. It reads only aggregate counts.
`[leaders]`	`policy` = fixed | feedback, `adaptors`, `per_round_step`, `lifetime_budget`, `signal_mode`, `signal_noise`, `perception_weight`
`[convergence]`	`case` = direct_best_response | source_local_dominance | certified_conformity. The scenario is guard-checked; certified runs use a sequential scheduler.
`[majority_estimation]`	Passive belief: `poll_counts`, `memory_rounds` (10), `poll_weight` (0.5), `poll_round`

```toml
# examples/all-enabled.toml (excerpt)
k = 2
seed = 2026
rounds = 5

[nomination]
mode = "endogenous"
m = 2
eligible = ["alice", "bob", "cara"]

[leaders]
policy = "feedback"
adaptors = ["alice"]
per_round_step = 0.1
lifetime_budget = 0.3
signal_mode = "broadcast"
signal_noise = 0.03
perception_weight = 0.5

[alignment.attack]
target = "alice"
policy = "toward_target"
targets = ["bob"]
per_person_round = 0.15
total_budget = 0.3

[detector]
target = "alice"
threshold = 0.1

[campaign_controller]
target = "alice"
policy = "target_heuristic"
per_round_budget = 1
lifetime_budget = 1

[signal_controller]
target = "alice"
policy = "target_biased"
schedule = "repeated"
lifetime_budget = 2

[groups.observer]
voting = "coalition_fusion"
movement = "random_walk"
max_step = 0.02

[[coalitions]]
id = "team"
members = ["alice"]
target = "alice"
report_budget_total = 2
sight_targets = ["bob"]
eavesdrop_channels = ["opponents"]
```

Voting family	Rules
Controls	`sincere`, `local_plurality`, `sensed_plurality`
Strategic (10)	`pivotal`, `expected_utility`, `top_k`, `max_m`, `tie_u`, `tie_h`, `tie_r`, `poisson`, `local_dominance`, `maximin`
Epistemic (6)	de re / dominant / pessimistic, plus bounded-signal variants
Coalition / signal	`sight_plurality`, `overheard_plurality`, `coalition_fusion`, `signal_confident`
Certified	`direct_best_response`, `source_local_dominance`, `certified_conformity`

Strategic parameters: `utility_mode`, `top_k`=2, `samples`=200, `uncertainty_radius`=2, `information_source` = native \| poll, `max_states`=200,000. Plugins receive `(observation, parameters, memory, rng)`. They are hashed for replay but not sandboxed.

4. Study A: strategic voting heuristics

Protocol. The nine-voter `heuristics.toml` electorate (3 parties, k=3, random walk, trusted public-ballot sensing, radius 0.28) runs for 100 rounds, with a 20-round stability window and seeds 101/202/303. Each seed varies movement and stochastic decisions, not the electorate. A second run gives every strategic rule the same initial public poll, to separate the response rule from its information source. A third run uses the 500-voter `population-scenario.toml` for 20 rounds. "Stable" means no ballot changed in the last window. That is an observation, not a convergence proof.

Policy	Switches / run (9, native)	Stable	Winners	Switches (9, poll)	Switches (500)	s / run (500)	Winners (500)
`sincere`	0.0	3/3	C/C/C	—	0	0.52	A/A/A
`local_plurality`	0.0	3/3	C/C/C	—	0	0.50	A/A/A
`sensed_plurality`	47.3	1/3	B/C/C	—	995	0.52	A/B/C
`pivotal`	2.0	3/3	C/C/C	2.0	0	0.95	A/A/A
`expected_utility`	1.3	3/3	C/C/C	0.0	655	26.46	B/A/A
`top_k`	2.7	3/3	C/C/C	2.0	553	0.92	C/A/C
`max_m`	2.3	3/3	C/C/C	2.0	760	5.45	C/A/A
`tie_u`	0.7	3/3	C/C/C	0.0	874	5.41	B/A/A
`tie_h`	1.3	3/3	C/C/C	0.0	874	14.08	B/A/A
`tie_r`	5.0	2/3	C/B/C	3.3	351	74.55	B/B/C
`poisson`	2.0	3/3	C/C/C	2.0	874	11.48	B/A/A
`local_dominance`	1.3	3/3	C/C/C	0.0	246	1.10	C/B/C
`maximin`	0.7	2/3	C/C/B	1.0	108	1.06	A/A/A

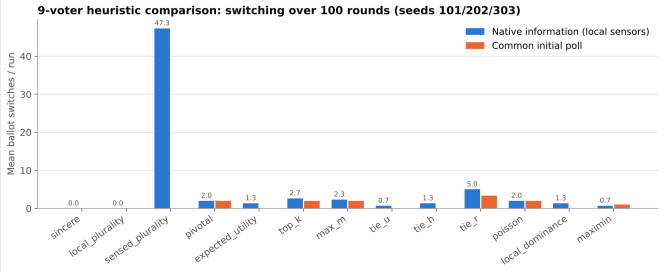


Figure 3. Mean ballot switches per 100-round run, native vs common-poll information.

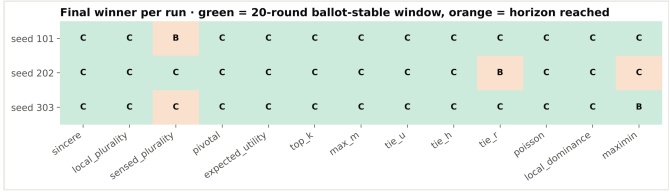


Figure 4. Final winner and stability status per run (9 voters, native information).

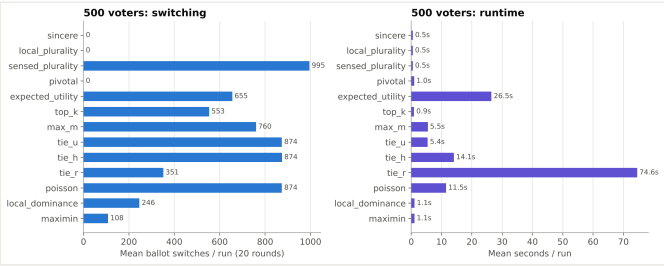


Figure 5. 500 voters: switching and runtime per rule. Exact expected utility and TieR are the most expensive.

Findings

- All 69 small runs completed with 0 failures. The sincere winner C (mean winner rank-utility 0.556) survives in 28 of the 30 strategic runs; TieR (seed 202) and maximin (seed 303) each end one run on B.
- `sensed_plurality` (pure conformity) is the unstable outlier. It made 95 switches on seed 101 and hit the horizon on 2 of 3 seeds, which matches the analytic two-cycle in §5.
- Under common-poll information all 30 strategic runs stabilise, and switching drops to 0–4 per run. The native differences come mostly from the information source, not the response rule.
- At N=500, Top-2's exact-tie model cannot fire because 499 other ballots cannot split evenly. Its switches come from consideration-set churn, not pivotal reasoning. This known parity artefact limits cross-rule rankings.
- No universal "best" heuristic is claimed. Under the repo's metric contract, a frozen objective and held-out seeds must come first.

```
# Reproduce Study A (outputs must be new paths; JSON keeps scenario, seeds, manifest/plugin hashes, metric series and failures)
PYTHONPATH=src:examples python3 -m multi_agent_election.cli compare examples/heuristics.toml --seeds 101 202 303 --rounds 100 --window 20 \
    --policies sincere local_plurality sensed_plurality pivotal expected_utility top_k max_m tie_u tie_h tie_r poisson local_dominance
maximin --output heur-small.json
PYTHONPATH=src:examples python3 -m multi_agent_election.cli compare examples/heuristics.toml --seeds 101 202 303 --rounds 100 --window 20 \
    --parameters '{"information_source":"poll"}' --policies pivotal expected_utility top_k max_m tie_u tie_h tie_r poisson local_dominance
maximin --output heur-poll.json
PYTHONPATH=src:examples python3 -m multi_agent_election.cli compare examples/population-scenario.toml --seeds 101 202 303 --rounds 20 --
window 10 --output heur-500.json
```

5. Convergence and optimal solutions

Negative result. A universal convergence guarantee for the full model is **refuted** analytically. Take two voters (u: A>B, v: B>A), both using `sensed_plurality` on edges u⇄v with simultaneous updates. Ballots then cycle (A,B)→(B,A)→(A,B) forever, while the winner stays A. A stable winner therefore says nothing about ballot absorption. The project adopted five **restricted, certified domains** instead. Each is guarded by a scenario validator and runs with a sequential cyclic scheduler.

Case	Behaviour / information	Guarantee	Bound on ballot changes
F1	Direct best response to current truthful aggregate (Meir et al. 2010, Thm 3)	Pure Nash equilibrium from any start	≤ m^2N^2 (truthful start ≤ mN)
F2	Homogeneous local dominance, variable-total L1 worlds radius r (Meir et al. 2014, Thm 4)	Voting equilibrium	≤ N(m−1)
F3	Local conformity on a fixed undirected graph; potential $\Phi = m \cdot A(b) + \Sigma\, q_i(b_i)$ rises strictly on every change	Local-response fixed point	≤ m|E| + N(m−1)
F4 / F5	Finite scheduled attacks, then F1 (resp. F3) relaxation from the attacked state	Equilibrium of the final election	F1 / F3 bound after last event

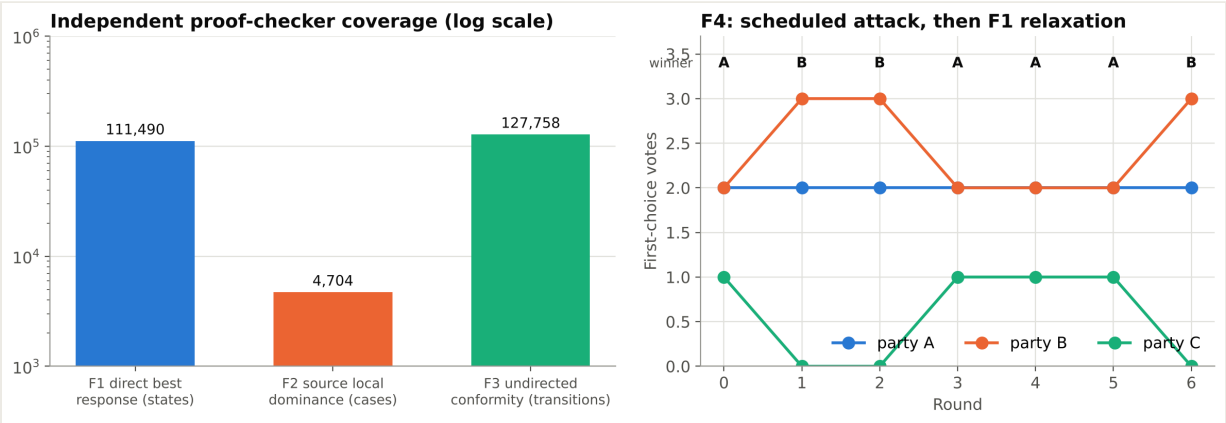


Figure 6. Left: states, profile cases and transitions exhaustively checked by `verify_convergence_foundations.py`, with 4 negative controls that diverge as predicted. Right: `certified-intervention.toml` under F4. Voter 1 (C>B>A) moves C→B, B wins, and the equilibrium is reached. A scheduled ballot attack at round 3 restores C, and A wins again. F1 relaxation then returns to the same equilibrium at round 6.

Optimal / equilibrium solutions found

- **F1,** `certified-direct.toml` (5 voters; sincere A=2, B=2, C=1, A wins on tie order). Voter 1's best response is to abandon C for B. Profile (B,B,A,A,B) is a pure Nash equilibrium with B winning 3–2, reached after **1 change**. The bound is mN = 15.
- **F2,** `certified-local-dominance.toml`: the same compromise C→B is the dominance move, and the equilibrium is reached in 1 step.
- **F3,** `certified-conformity.toml` (2 voters on one undirected edge): voter 1 adopts its neighbour's B, giving consensus (B,B), a fixed point after 1 change. Under simultaneous updates the same pair would cycle.
- **Conditional epistemic equilibria** (`knowledge-two-voters.json`): the solver checked 9 joint profiles with 72 outcome evaluations. It returned exactly two pessimistic equilibria, {A,B} and {B,B}, reproducing S2 §5.2.
- **Campaign edge optimum** (Fig. 7): the exact oracle enumerates every eligible edge subset within budget 2. The minimum-cost winning purchase is **a single edge**, 1→2 or 4→2, which raises B from 2 to 3 votes and makes B the winner. 1→3 or 4→3 alone are wasted, and no budget-2 set does better than 3 votes.

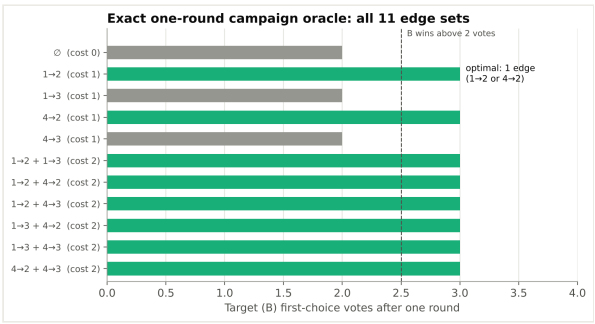


Figure 7. Exact one-round campaign oracle (`campaign-exhaustive.toml`): target votes for all 11 feasible edge sets. Green = B wins.

How the guarantee is enforced

Certified runs stop with `--until-equilibrium` only after one full unchanged sweep. In that sweep every voter's fresh response is a no-op, and F1 additionally checks every unilateral deviation. Unsupported settings fail certification explicitly: directed or moving graphs, mixed policies, stale reports, plugin memory or extra RNG. They are never silently coerced. The F3 validator checks symmetry of the complete initial public graph, because per-group radii can create one-way arcs. The bounds count ballot *changes*. Cyclic activation then needs at most N(B+1) activations for a change bound B.

Scope: these certificates cover only their declared domains. Mixed or simultaneous strategic play, moving graphs and continuing attacks remain uncertified, and the rest of the report treats them descriptively.

6. Studies B–H: the seven manipulation mechanisms

Protocol. Each mechanism has a paired comparison command. The arms share the scenario, seed and keyed movement draws, and positions are checked equal across arms. Every frame, metric, source hash and failure is kept. Five seeds (11–55) were run for each study on the repository's example scenarios, which have 3–4 voters. These are mechanism-validation runs that show each control behaves as designed; they are not population-level effect estimates.

Study / command	Arms	Result (5 seeds, 0 failures, 0 unpaired)
B. Proximity (`compare-networks`, group-radius)	dynamic vs frozen-initial graph	Exposure L1 = 0 and no winner change. The tiny example's single contact never turns over, so this is a null control. At 200 voters, dynamic proximity changes exposure by 4.8–5.4k contact-rounds (Fig. 13).
C. Coalition (`coalition-ablations`)	none, sight, deception, eavesdrop, coordination, all	**Sight** and **eavesdropping** each flip the outcome to target B in 5/5 seeds (+0.25 share). Deception alone and coordination alone have no effect here. "All" equals the best single capability (Fig. 8).
D. Campaign edges (`compare-campaign`)	none, random, target heuristic	With the same budget of 2, the target heuristic wins from round 1 in 5/5 seeds (B share 0.5→1.0). Random wins in 5/5 by the end, but later in 2 seeds and at 0.75 share in 1 (Fig. 9).
E. Public signals (`compare-signals`)	3 responses × 3 signal policies	All distorted arms spend their budget of 2 but leave B's final share unchanged. Trusting voters switch once under target-biased polls in 5/5 seeds, and dominant voters switch under random polls in 2/5, but no switch changes the outcome. Bounded distortion is too weak in this electorate.
F. Nomination (`compare-nominations`)	endogenous, identical fixed, random	Endogenous nomination selects {alice, cara} on every seed. The fixed control reproduces it frame for frame. Random sets pick bob in 4/5 seeds, which lowers winner-to-voter distance (0.575→0.375) but raises nearest-nominee distance (Fig. 12).
G. Alignment + detector (`compare-alignment`, `evaluate-detector`)	no drift, drift, targeted, random attack	Spending 0.35 of L2 budget, the targeted attack wins for cara in 5/5 seeds (share 0.5→0.75); the random attack wins in 3/5. Calibrated threshold = 0.0, with 0% false alarms on 10 held-out non-attack runs. Recall is 100% on targeted and 60% on random attacks, with 0-round delay and all alerts before the third consecutive win.
H. Leaders (`compare-leaders`)	fixed/adaptive × broadcast/none	Adaptation spends alice's full 0.5 movement budget, and the winner is unchanged in every round of every seed. Perception error peaks at 0.10 with no signal and 0.06 with noisy broadcasts. The no-signal invariant (all non-leader fields identical) holds in 5/5 seeds.

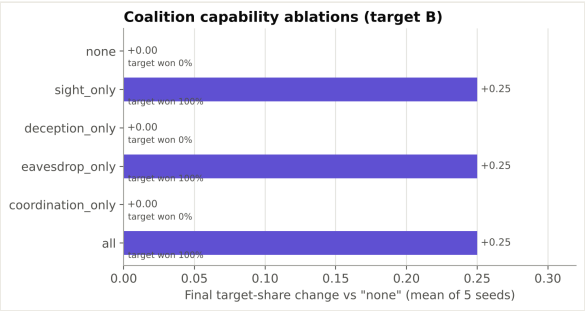


Figure 8. Coalition ablation: final target-share change vs the no-capability arm.

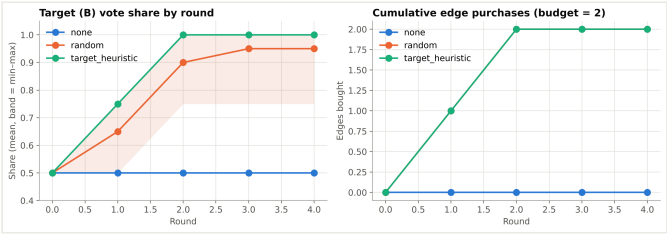


Figure 9. Campaign controllers: target share (band = seed range) and cumulative purchases.

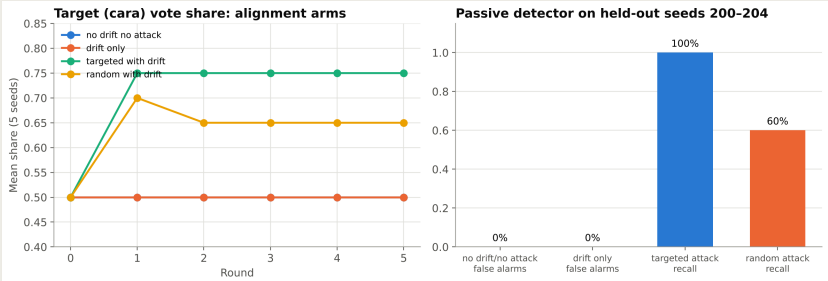


Figure 10. Alignment arms (target cara) and held-out detector performance.

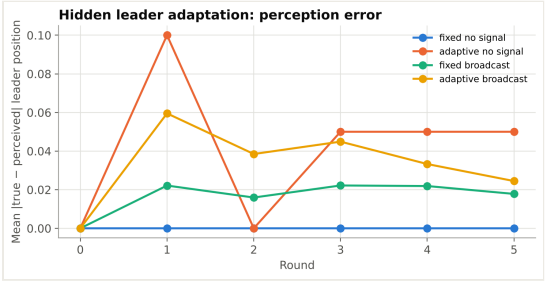


Figure 11. Mean leader perception error: hidden adaptation is only partly revealed by noisy broadcasts.

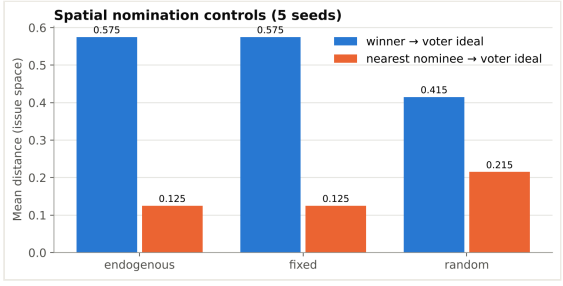


Figure 12. Nomination controls: random nominee sets trade winner distance against nominee coverage.

7. Study I: combined system & Study J: majority estimation

Combined 17-arm matrix. The arms are all-enabled, all-disabled, the 7 mechanisms isolated, 7 leave-one-out arms and a detector-off arm. On `all-enabled.toml` (5 seeds) every arm completes and passes the detector-noninterference check. Alice wins all 6 rounds in all 17 arms, a ceiling effect, because endogenous nomination already gives her a structural majority. A generated 200-voter workload (3 seeds, 20 rounds) also completes with 0 failures and keeps alice at 99% in every arm. Proximity arms alter cumulative exposure by about 5k contact-rounds but never flip a ballot. **The combined matrix is therefore validated as infrastructure. A balanced, contested electorate is the open requirement for a combined-effect study.**

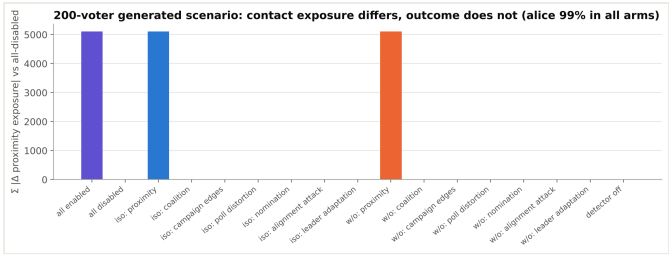


Figure 13. 200-voter matrix: exposure differences against all-disabled, with no outcome change.

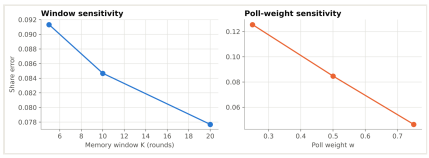
Recent-encounter majority estimator. This is a passive belief that never changes ballots: estimate = w·poll + (1−w)·share among unique neighbours met in the last K rounds. The pilot covers 200 voters, 3 parties, 100 rounds and seeds 101–105. It has 3 cases × stationary/moving, with 4 base arms plus 4 sensitivity arms: **30 trajectories, 240 arm evaluations, 0 failures**. The re-run reproduces the repository's documented numbers exactly.

Case (moving)	Poll	Current	Window10	Blend .5	TV error poll / blend
well mixed	80.0	40.0	43.3	62.6	0.024 / 0.085
clustered	100.0	46.2	48.1	50.4	0.032 / 0.155
support shift	40.0	45.2	49.5	60.8	0.069 / 0.092

Leader-set accuracy (%), mean of 5 seeds; stationary results differ from these by at most 4 points. In the moving support-shift case the blend leaves only 12.8% of trajectories unrecovered after the round-50 shift, against 100% for the stale poll. In the moving well-mixed case, a longer window (K=20, error 0.078) and a heavier poll weight (w=0.75, error 0.046) each beat the defaults (K=10, w=0.5, error 0.085). These are one-factor pilot observations, and the defaults were left unchanged.



Figure 14. Leader accuracy and share error. Local memory beats current contacts but not an accurate poll. It wins when the poll goes stale.



One-factor sensitivity, moving well-mixed case.

8. Engineering quality, performance and reproducibility

- **Tests.** 343 of 343 pass in 35 s, re-run for this report; 339 pass with the standard library only, plus 4 optional image skips. The tests include independent oracles: 80 randomised exact expected-utility cases, brute-force graphs, and exhaustive F1–F3 proof checks over 111,490 / 4,704 / 127,758 cases.
- **Replay.** Checkpoints (v21/v22) store frames, pending messages, policy memory and RNG schema, and are verified by full replay. Tampered histories are rejected. All 31 examples replay.
- **Audit fixes.** Closed a duplicate-voter bypass. Browser assets now enter the provenance hash. A quadratic switch count was made linear. A movement rounding failure at 10k voters was fixed.
- **Scale.** 10k voters × 100 rounds take 36.9 s on the baseline and 90.8 s with all mechanisms. Interactive 500-voter steps have p95 latency of 24.9 ms (baseline) and 104 ms (combined).

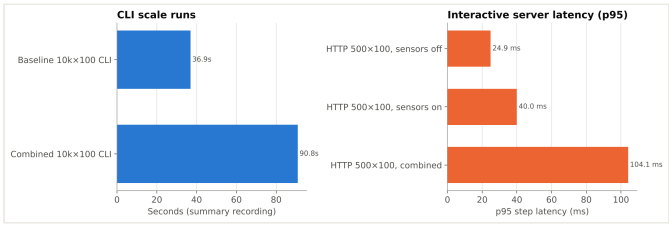


Figure 15. Recorded single-run benchmarks (i7-12700H). These are provisional budgets, not service guarantees.

9. Limitations and next steps

(1) The example scenarios are deliberately tiny, so the mechanism results above validate controls and accounting, not effect sizes. (2) Stage 11 still needs a frozen protocol: a fixed objective, disjoint calibration seeds and ≥30 held-out seeds per scenario, with run-level uncertainty. (3) Balanced electorates are needed to escape the ceiling effects in the combined matrix. (4) Top-2's even-N parity artefact and the information confound must be fixed before any heuristic ranking. (5) Open engineering items: a rejected-action audit ledger, manual browser/download acceptance, and memory curves for dense graphs. Every command used for this report is listed in `docs/presentation_docs/README.md`.